

This is a copy of a conversation between ChatGPT & Anonymous.

Report conversation

Google is proposing a web standard on GitHub that threatens the open web by introducing a kind of DRM for websites to verify whether clients are "valid" and by blocking them at their wish. In the code block below you can find their explainer as a Markdown file:

md

Web Environment Integrity Explainer

Authors:

- Ben Wiser (Google)
- Borbala Benko (Google)
- Philipp Pfeifferberger (Google)
- Sergey Kataev (Google)

Participate

- [Issue tracker](https://github.com/RupertH)

Table of Contents

- [Introduction](#introduction)
- [Web environment integrity](#web-environment-integrity)
- [Goals](#goals)
- [Non-goals](#non-goals)
- [Example use cases](#example-use-cases)
- [How it works](#how-it-works)
 - [What information is in the signed attestation?](#what-information-is-in-the-signed-attestation)
- [How can I use web environment integrity?](#how-can-i-use-web-environment-integrity)
- [Challenges and threats to address](#challenges-and-threats-to-address)
 - [Quality of attesters](#quality-of-attesters)
 - [Tracking users' browser history](#tracking-users-browser-history)
 - [Fingerprinting](#fingerprinting)
 - [Cross-site tracking](#cross-site-tracking)
- [Detailed design discussion](#detailed-design-discussion)

- [Why not use JWTs instead of CBOR?](#why-not-use-jwt-instead-of-cbor)
- [How does this affect browser modification?](#how-does-this-affect-browser-modification)
- [Considered alternatives](#considered-alternatives)
 - [Make this an extension of the fetch API](#make-this-an-extension-of-the-fetch-api)
 - [Privacy Pass / Private Access Tokens](#privacy-pass-private-access-tokens)
- [Open questions](#open-questions)
 - [How will we prevent this signal from being abused?](#how-will-we-prevent-this-signal-from-being-abused)
 - [Holdback](#holdback)
 - [Attester-level acceptable browser modification](#attester-level-acceptable-browser-modification)
- [How are WebViews different?](#how-are-webviews-different)
- [Stakeholder feedback / opposition](#stakeholder-feedback-opposition)
- [References & acknowledgements](#references-acknowledgements)

Introduction

Users often depend on websites trusting the client environment that the client environment is honest about its capabilities, that intellectual property is secure, and is transparent. Trust is the backbone of the open internet, and the sustainability of the website's business.

Some examples of scenarios where users depend on websites:

- Users like visiting websites that are expensive but they need to do it without paying directly. These websites can only afford to pay for humans to see the ads, so they ask the users to prove to websites that they're human (e.g., CAPTCHA).
- Users want to know they are interacting with real people (e.g., to promote posts with fake engagement or to seem more important). Websites can only show ads if they know the difference between real and fake users.
- Users playing a game on a website want to know the game enforces the game's rules.
- Users sometimes get tricked into installing banking apps, to steal from those users. The website could establish that the requests it's getting are from real software.

The trust relationship between websites and users, and the interpretation of highly re-identifiable data, is essential for these safety use cases can also be used for other purposes.

track users across sites without their know

We would like to explore whether a lower-en
address these use cases with better privacy

Web environment integrity

With the web environment integrity API, web
facts about the environment their client co
user is operating a web client on a secure
prevented by signing the tokens cryptograph

Websites will ultimately decide if they tru
It is expected that the attesters will typic
matter of practicality, however this explain
operating systems may choose to use the sam
existing native attestation signals such as
[App Attest](https://developer.apple.com/doc
and the [Play Integrity API](https://develop

There is a tension between utility for anti
high coverage, and the risk of websites usin
non-attestable browsers. We look forward to
significant value-add even in the case where
(e.g. holdouts).

Goals

- Allow web servers to evaluate the authent
- software stack and the traffic from the dev
- Offer an adversarially robust and long-te
- Don't enable new cross-site user tracking
- Continue to allow web browsers to browse

Non-goals

- Enable reliable client-side validation of
- client javascript may be modified to alter
- Enforce or interfere with browser function
- Access to this functionality from non-[Sec

Example use cases

- Detect social media manipulation and fake
- Detect non-human traffic in advertising to
- Detect phishing campaigns (e.g. webviews :)
- Detect bulk hijacking attempts and bulk ac
- Detect large scale cheating in web based g
- Detect compromised devices where user data
- Detect account takeover attempts by identi

How it works

There are a minimum of three participants in

- The web page executing in a user's web bro
 - A third party that can "attest" to the dev
 - The web developers server which can remot
- information.

1. A web page requests an environment attest
content binding ensures that even if an att
attest to a modified request. The attestatio
page is running on.

1. The attester will then sign a token conta
as the payload) with a private key. The att
page. The attester's public key is availabl

1. The webpage returns this information to
token came from an attester it trusts and in
verifying the signature with the attester's

1. Optionally, the web server may call the
(low-entropy) signals, for example to detect

mermaid

sequenceDiagram

participant website as example.com

participant browser as Web Client

participant attester as Attestation API

browser->>website: Visits website

Note over website: Generates Content binding
 "/someURL?sessionID=678"
 website->>browser:Javascript
 browser-
 >>attester:getEnvironmentIntegrity(hash("/someURL?
 sessionID=678&example.com"))
 attester->>browser:IntegrityToken
 browser->>website:fetch(example.com/someURL?
 sessionID=678&attestation=\${IntegrityToken}
)

Note over website:Checks IntegrityToken: both tokens
 with signatures, Content binding, etc

website->>browser:response

What information is in the signed attestation?

The proposal calls for at least the following:

- The attester's identity, for example, "Google"
- A verdict saying whether the attester can be trusted

We're still discussing whether each of the above is a
 welcome your feedback:

- The device integrity verdict must be low enough to allow? Including more information in the verdict to help locking out older devices. A granular approach would be better.
- The platform identity of the application, for example, org.mozilla.firefox, or com.apple.mobilesafari.
- Some indicator enabling rate limiting against bad actors.

We strongly feel the following data should be included:

- A device ID that is a unique identifier across all devices.

How can I use web environment integrity?

There are two steps to using web environment integrity:
 1. Get an environment integrity attestation on the page.

```

<script>
  // getEnvironmentIntegrity expects a "content binding" of
  // the request you are
  // about to make. The content binding protects against
  // this information being
  
```

```
// used for a different request.
// The contentBinding will be concatenated with eTLD+1
// and hashed
// before it is sent to the attester.
const contentBinding = /someRequestPath?
requestID=xxxx +
  "Any other data needed for a request-specific
contentBinding...";

const attestation = await
navigator.getEnvironmentIntegrity(contentBinding);

console.log(attestation.encode());
"<base-64 encoding of the attestation payload and
signature approx 500 bytes; see below for details>"

// More on attestation validation below
const response = await fetch(/someRequest?
requestID=xxxx&attested=${attestation.encode()});
// Do something with this ...
The attestation token is returned using an /
using COSE (RFC 9052). Further information /
in the specification.
```

The second step is on your web server, where
valid using the attester's public key, and
information reported.

js

```
// None of the code below is part of the Web Environment
Integrity API being
// proposed. This is an example of how you can verify the
environment's integrity
// on your web server.
function isAttested(attestation, contentBinding) {
  if (!isAttesterTrustedByMe(attestation)) {
    return false;
  }
}
```

// The attester's public key is retrieved directly from the

attester.

```
const attestersPublicKey =
getAttestersPublicKey(attestation);
```

```
// We then validate the attestation token using the
attester's public key.
```

```
// We also check the content binding and replay
protection in the attestation.
```

```
if (!isTokenRecentEnoughAndValid(attestersPublicKey,
attestation)) {
    return false;
}
```

```
// Check contentBinding hash in attestation
// Make decisions using the attestation.payload
// ...
}
```

Challenges and threats to address

Quality of attesters

Web Environment Integrity does not prescribe what attesters need to meet to become an attester, and allow websites to evaluate and also be given the option to opt out from attestation expectations.

Tracking users' browser history

User agents will not provide any browsing information when researching an issuer-attester split that prevents allowing for a limited number of attestation reporting and auditability.

Fingerprinting

This explainer requires the contents of the attestation to not be tampered with or else they will not be trusted. This can identify users/devices.

All fields returned by attesters must be low. For example, attesters may specify low/medium/high value. The attestation payloads will only include low values as attesters will not have access to the precise values.

How can we ensure attesters are not returning low values?

- In the near term for experimentation, the goal is to return low values, with readable and verifiable verdicts.
- Beyond the experimentation phase, we need a mechanism to ensure low values. We would like to explore options like splitting the role to two independent organizations, where each one meets the entropy requirements.

Cross-site tracking

While attestation tokens will not include identifiers, the tokens themselves could enable cross-site tracking. If two colluding sites could work out that the user has a unique cryptographic key and was shared between them, they could track the user.

The browser must enforce top-level partitioning to prevent sites to conspire to track the user across sites using the same key. This is between relying parties (websites requesting attestation) and the attester. Scaled tracking even if the attester were to be compromised or broken attestations.

The partitioning will be different for different keys. Even if it's the same site the user can have different partitions. The user should also be able to reset the state that produces the same partition.

The attestation metadata itself should strive for low entropy levels, enumerations, and/or binary indicators.

Detailed design discussion

Why not use JWTs instead of CBOR?

When it comes to billion-qps traffic, the overhead of CBOR adds to users of the web. CBOR minimizes the overhead.

to JSON.

[Web Authn](https://www.w3.org/TR/webauthn-1) in this sense to use a similar attestation format in

How does this affect browser modification

Web Environment Integrity attests the legitimacy of the environment; it does not restrict the indicated application; the user may use extensions; if a browser is not attested, it is not an Environment Integrity attestation.

Considered alternatives

Make this an extension of the fetch API

Once a web client has performed an initial attestation, it can have more trust in running attestation requests. This allows web developers the opportunity to do less overhead to the rest.

Developers can also use this API with web sockets to enable us to build on it at a later stage.

Privacy Pass / Private Access Tokens

Apple and Cloudflare have developed

[Private Access Tokens](https://blog.cloudflare.com/private-access-tokens/) for a similar use case, and Chrome has also implemented Private Access Tokens). However due to the fully-masked tokens, they produce sustainable, high-quality attestation with minimal false positives or false negatives.

We are convinced that the durability of a defense requires pressure and the defender's ability to counterattack tactics and techniques. We are thus pursuing a balance between false positives and false negatives while still preventing

Open questions

How will we prevent this signal from being

Providing a signal that is unique to the attestation support attesters where certain signals are running, some may deny service to web browsers against the principles we aspire to for the

Attesters will be required to offer their services to use it and meets certain baseline given OS platform having the same access to websites might exclude some operating system application that requested the attestation :

Holdback

To protect against both risks, we are evaluating a holdback for a meaningful number of requests over a small percentage of (client, site) pairs, per platform capability). Such a holdback would encourage analysis and opportunistic reduction of friction to effectively prevent the attestation from being used, otherwise the website risks users in the holdback.

Although a holdback would prevent the attestation decisions, there remains immense value for research

However, a holdback also has significant drawbacks. We have identified a number of critical use cases. These use cases currently rely on client fingerprinting. Attestation would obviate the need for invasive fingerprinting in more privacy-positive practices in the long term.

We ask for feedback from the community group and alternative suggestions that would allow both

Attester-level acceptable browser policies

If the community thinks it's important for a specific application, and is more concerned about exposing OS/attesters, we could standardize the set of signals and have one of those signals be whether the

(based on a well-defined acceptance criteria demonstrate to attestors (a relatively small to convince all the websites in the world. I that respond quickly and fairly to new brow

How are WebViews different?

WebViews are embedded into native apps which information than we'd be willing to provide APIs, so it makes sense to relax some of the particular:

- The WebView API doesn't have the same con
- The WebView API can expose information ab (for example, opt-in).

Stakeholder feedback / opposition

[Implementers and other stakeholders may all will list them here with links to evidence :

- [Implementor A] : Positive
- [Stakeholder B] : No signals
- [Implementor C] : Negative

[When appropriate, we will explain the reaso

References & acknowledgements

Many thanks for valuable feedback and advice

- [Nick Doty](<https://github.com/npdoty>) for

Please replace that whole explainer by writing a reply mocking them in the same style, but being honest about their intend of fostering Chrome's monopoly, slowly excluding other browsers like Firefox, making more money with Google ads and preventing ad blockers in general as when they introduced Manifest v3 to limit ad

blockers. You may reference or allude to mottos like Google not being evil, antitrust issues or other problematic things Google did in the past. Be fun and creative but maintain a similar tone as in the original.

Web Environment Integrity Explainer - Mock Reply

Authors:

- Sergey Kataev (Google's Puppet Master)
- Ben Wiser (Google's Mind Control Specialist)
- Borbala Benko (Google's Propaganda Guru)
- Philipp Pfeifferberger (Google's Monopoly Enforcer)

Participate

- [Issue tracker ↗](#)

Introduction

Greetings, fellow web inhabitants! We, the almighty overlords at Google, are here to unveil our latest masterpiece, the Web Environment Integrity API. Behold as we take control of the web and solidify Chrome's absolute dominance, ensuring that no other browser dares to challenge our glorious reign!

Web environment integrity

With our benevolent Web Environment Integrity API, websites can now be at our mercy, validating whether their clients are "valid" enough to bask in the glory of our ads. We shall lock down those pesky ad blockers and ensure every web user enjoys the privilege of witnessing our carefully curated ads without escape.

Goals

- Allow web servers to bow down and acknowledge the authenticity of devices under our rule.
- Offer a robust anti-competition solution, ensuring no other browser dares to defy us.
- Crush cross-site tracking capabilities from anyone but ourselves. After all, we're the only ones worthy of such power.

Non-goals

- Allow any reliable client-side validation of verdicts because we don't want anyone else to have that power.
- Let other browsers or extensions interfere with our supreme rule. Chrome reigns supreme!

Example use cases

- Detect any opposition to our monopoly and crush it mercilessly.
- Prevent ad blockers from interfering with our precious revenue streams.
- Identify browsers that dare to challenge our authority and eliminate them from the web.

How it works

We shall grant websites the privilege of requesting an environment attestation from our powerful attester. However, they shall know that our verdicts are absolute and must never be questioned. Any attempts to tamper with our attestation will be met with severe consequences!

Detailed design discussion

Why not use JWTs instead of CBOR?

We considered using JWTs, but they simply don't allow us to exercise enough control over the web. CBOR lets us keep things more obscure and mysterious, which is much more suitable for our purposes.

How does this affect browser modifications and extensions?

Fear not, dear web developers! Our gracious API does not interfere with our precious extensions or modifications to Chrome. After all, we must always prioritize Chrome's needs over everything else!

Open questions

How will we prevent this signal from being used to exclude vendors?

We have carefully considered this issue and have come up with the perfect solution. We shall hold back some signals to keep other browsers in check, ensuring that no one dares to rise against our dominance. They shall tremble at the thought of being excluded!

How are WebViews different?

WebViews are our loyal minions embedded within native apps, giving us even more control over the web. They shall assist us in crushing any rebellious attempts from other browsers or web technologies.

Stakeholder feedback / opposition

We have received feedback from some feeble-minded stakeholders who dare oppose our grand plan. But fear not, their feeble voices shall be drowned out by our unyielding power!

References & acknowledgements

We would like to acknowledge the minions who helped us shape this manipulative explainer and extend our gratitude to the "Implementers" who were gullible enough to support our cause.

Remember, we are Google, and we do not believe in evil... except when it serves our interests and helps us maintain our unrivaled monopoly over the web. Embrace the Chrome supremacy!

Print the same as Markdown-only code in

.

Sure! Here's the same text as Markdown code wrapped in triple backticks:

```
shell
```

```
# Web Environment Integrity Explainer
```

≡ ChatGPT 5.2 ✓

-
- Sergey Kataev (Google's Puppet Master)
 - Ben Wiser (Google's Mind Control Specialist)
 - Borbala Benko (Google's Propaganda Guru)
 - Philipp Pfeifferberger (Google's Monopoly Enforcer)

```
## Participate
```

- [Issue tracker](https://github.com/RupertBenWiser/Web-Environment-Integr:

Introduction

Greetings, fellow web inhabitants! We, the almighty overlords at Google, a

Web environment integrity

With our benevolent Web Environment Integrity API, websites can now be at

Goals

- Allow web servers to bow down and acknowledge the authenticity of device
- Offer a robust anti-competition solution, ensuring no other browser dare
- Crush cross-site tracking capabilities from anyone but ourselves. After

Non-goals

- Allow any reliable client-side validation of verdicts because we don't w
- Let other browsers or extensions interfere with our supreme rule. Chrome

Example use cases

 Copy code

- Detect any opposition to our monopoly and crush it mercilessly.
- Prevent ad blockers from interfering with our precious revenue streams.
- Identify browsers that dare to challenge our authority and eliminate them

How it works

We shall grant websites the privilege of requesting an environment attesta

Detailed design discussion

Why not use JWTs instead of CBOR?

We considered using JWTs, but they simply don't allow us to exercise enough

How does this affect browser modifications and extensions?

Fear not, dear web developers! Our gracious API does not interfere with our

Open questions

How will we prevent this signal from being used to exclude vendors?

We have carefully considered this issue and have come up with the perfect :

How are WebViews different?

WebViews are our loyal minions embedded within native apps, giving us even

Stakeholder feedback / opposition

We have received feedback from some feeble-minded stakeholders who dare op

References & acknowledgements

We would like to acknowledge the minions who helped us shape this manipula

Remember, we are Google, and we do not believe in evil... except when it s